LanguageName: PLPDI Mini-Project

David Tilman
Student ID: 2763839

October 17, 2026

Repository: `https://github.com/DavidTilman/plpdi-mini-project`
Commit: `TODO`

1 Informal Description

2 Literature Review

3 Formal Syntax and Semantics

3.1 Syntax

$\oplus$	::=	$+ \mid - \mid * \mid / \mid \% \mid < \mid <= \mid == \mid != \mid >= \mid > \mid \mid\mid \mid \&\&$	binary operators
τ	::=	`Nat` $\mid$ `Int` $\mid$ `Real` $\mid$ `Bool`	base types
	$\mid$	$(\tau_1, \ldots, \tau_n) \to \tau$	function type
	$\mid$	R	record type
e	::=	n	literal
	$\mid$	`True` $\mid$ `False`	bool literal
	$\mid$	x	variable
	$\mid$	$-e$	unary negation
	$\mid$	$e_1 \oplus e_2$	binary operation
	$\mid$	$e.f$	field access
	$\mid$	$e(e_1, \ldots, e_n)$	application
	$\mid$	$R(e_1, \ldots, e_n)$	record construction
	$\mid$	`Lambda` $(x_1 : \tau_1, \ldots, x_n : \tau_n) : \tau \Rightarrow e$	lambda
	$\mid$	$e \ ? \ e_1 : e_2$	if expression
b	::=	$\{ \, s_1; \ \ldots; \ s_n; \, \}$	block ($n \geq 0$)
s	::=	`Var` $x : \tau = e$	declaration
	$\mid$	$x := e$	assignment
	$\mid$	`While` (e) b	loop
	$\mid$	`If` (e) b	conditional
	$\mid$	`If` (e) b_1 `Else` b_2	conditional with else
	$\mid$	`Ret` e	return
d	::=	`Record` R $(f_1 : \tau_1, \ldots, f_n : \tau_n)$	record declaration
	$\mid$	`Func` F $(x_1 : \tau_1, \ldots, x_n : \tau_n) : \tau = b$	function declaration
c	::=	$d \mid s$	top-level item
p	::=	$c_1; \ \ldots; \ c_n;$	program

`If` (e) b abbreviates `If` (e) b `Else` $\{ \}$.

3.2 Type System

3.3 Operational Semantics

Notation We write:

- $\overline{x_n}$ to mean $x_1, \ldots, x_n$, and $\overline{x : \tau}$ for $x_1 : \tau_1, \ldots, x_n : \tau_n$. $\rho[\overline{x} \mapsto \overline{\ell}]$ abbreviates $\rho[x_1 \mapsto \ell_1, \ldots, x_n \mapsto \ell_n]$, and $\overline{\ell} \notin \mathrm{dom}(\sigma)$ means the ℓ_i are distinct and none is in $\mathrm{dom}(\sigma)$.
- a value $v ::= n \mid \texttt{True} \mid \texttt{False} \mid R(\overline{v_n}) \mid \mathsf{clo}(\overline{x}, b, \rho)$. b in closure is a block body.
- an environment $\rho : \texttt{Var} \to \texttt{Loc}$ to mean a function that maps a variable name `Var` to a location `Loc`.
- that the store $\sigma : \texttt{Loc} \to \texttt{Value}$ maps a location `Loc` to a value `Value`.
- $f[a \mapsto b]$ to mean update f so that it maps a to b.
- $\mathsf{dom}(\sigma)$ to mean the set of locations that σ is defined on.
- the outcome of a *statement* $r ::= \mathsf{normal} \mid \mathsf{ret}\ v$, where normal is a statement that does not return nor throw an error, and $\mathsf{ret}\ v$ is a statement that returns a value v.
- $\hat{\oplus}$ to mean the result of a binary operation $\oplus$ on two values, v_1, v_2
- Δ for the *record table*, which maps each record name R declared in the program to its fields, so $\Delta(R) = (f_1 : \tau_1, \ldots, f_n : \tau_n)$. Δ is built once by P-PROG and never changes, so we leave it implicit in the judgements.

We write four kinds of judgement:

- Statements:

$$\langle s, \rho, \sigma \rangle \Downarrow \langle r, \rho', \sigma' \rangle$$

  A statement s, in environment ρ with store σ evaluates to an outcome r, an updated environment ρ' and store σ'.

- Blocks:

$$\langle b, \rho, \sigma \rangle \Downarrow \langle r, \sigma' \rangle$$

  A block b, in environment ρ and store σ evaluates to an outcome r and an updated store σ'.

- Expressions:

$$\langle e, \rho, \sigma \rangle \Downarrow \langle v, \sigma' \rangle$$

  An expression e in environment ρ and store σ evaluates to a value and updated store σ'

- Programs:

$$\langle p \rangle \Downarrow \langle \sigma' \rangle$$

A program p runs and ends with an updated store σ'

We begin by writing rules for evaluations.

$$\frac{}{\langle n, \rho, \sigma\rangle \Downarrow \langle n, \sigma\rangle}\ \text{E-NUM}$$

$$\frac{}{\langle \texttt{True}, \rho, \sigma\rangle \Downarrow \langle \texttt{True}, \sigma\rangle}\ \text{E-TRUE} \qquad \frac{}{\langle \texttt{False}, \rho, \sigma\rangle \Downarrow \langle \texttt{False}, \sigma\rangle}\ \text{E-FALSE}$$

$$\frac{\rho(x) = \ell}{\langle x, \rho, \sigma\rangle \Downarrow \langle \sigma(\ell), \sigma\rangle}\ \text{E-VAR} \qquad \frac{\langle e, \rho, \sigma\rangle \Downarrow \langle v, \sigma'\rangle}{\langle -e, \rho, \sigma\rangle \Downarrow \langle \hat{-} v, \sigma'\rangle}\ \text{E-NEG}$$

$$\frac{\langle e_1, \rho, \sigma\rangle \Downarrow \langle v_1, \sigma_1\rangle \qquad \langle e_2, \rho, \sigma_1\rangle \Downarrow \langle v_2, \sigma_2\rangle}{\langle e_1 \oplus e_2, \rho, \sigma\rangle \Downarrow \langle v_1 \hat{\oplus} v_2, \sigma_2\rangle}\ \text{E-OP}$$

$$\frac{\langle e, \rho, \sigma\rangle \Downarrow \langle \texttt{True}, \sigma'\rangle \qquad \langle e_1, \rho, \sigma'\rangle \Downarrow \langle v_1, \sigma''\rangle}{\langle e\ ?\ e_1 : e_2, \rho, \sigma\rangle \Downarrow \langle v_1, \sigma''\rangle}\ \text{E-COND-T}$$

$$\frac{\langle e, \rho, \sigma\rangle \Downarrow \langle \texttt{False}, \sigma'\rangle \qquad \langle e_2, \rho, \sigma'\rangle \Downarrow \langle v_2, \sigma''\rangle}{\langle e\ ?\ e_1 : e_2, \rho, \sigma\rangle \Downarrow \langle v_2, \sigma''\rangle}\ \text{E-COND-F}$$

$$\frac{\langle e, \rho, \sigma\rangle \Downarrow \langle R(v_1, \ldots, v_n), \sigma'\rangle \qquad \Delta(R) = (f_1 : \tau_1, \ldots, f_n : \tau_n) \qquad f = f_i}{\langle e.f, \rho, \sigma\rangle \Downarrow \langle v_i, \sigma'\rangle}\ \text{E-FIELD}$$

$$\frac{\langle e_i, \rho, \sigma_{i-1}\rangle \Downarrow \langle v_i, \sigma_i\rangle \quad (1 \le i \le n)}{\langle R(\overline{e}), \rho, \sigma_0\rangle \Downarrow \langle R(\overline{v}), \sigma_n\rangle}\ \text{E-RECORD}$$

$$\frac{}{\langle \texttt{Lambda}\ (\overline{x : \tau}) : \tau' \Rightarrow e, \rho, \sigma\rangle \Downarrow \langle \mathsf{clo}(\overline{x},\ \{\texttt{Ret}\ e;\},\ \rho), \sigma\rangle}\ \text{E-LAMBDA}$$

$$\frac{\begin{array}{c}\langle e, \rho, \sigma\rangle \Downarrow \langle \mathsf{clo}(\overline{x},\ b,\ \rho_\theta), \sigma_0\rangle \qquad \langle e_i, \rho, \sigma_{i-1}\rangle \Downarrow \langle v_i, \sigma_i\rangle \quad (1 \le i \le n) \\ \overline{\ell} \notin \mathrm{dom}(\sigma_n) \qquad \langle b, \rho_\theta[\overline{x} \mapsto \overline{\ell}], \sigma_n[\overline{\ell} \mapsto \overline{v}]\rangle \Downarrow \langle \mathsf{ret}\ v, \sigma'\rangle\end{array}}{\langle e(\overline{e}), \rho, \sigma\rangle \Downarrow \langle v, \sigma'\rangle}\ \text{E-APP}$$

E-NUM, E-TRUE, E-FALSE are trivial.

In E-VAR we see the first example using ρ and σ. In this rule we retrieve a memory location ℓ from ρ and then see that the return `Value` is the value at $\sigma(\ell)$.

In E-NEG and E-OP we see an example of $\hat{\oplus}$, where after evaluating the expressions on either side, we return the value of the operation applied between the two. Rules for each operation are defined in the appendix (TODO).

The E-COND rules are trivial.

In E-FIELD we require that e evaluates to a record with values $\overline{v}$. This can be achieved with E-VAR. We then require that the given record R has fields $f_1, \ldots, f_n$ and that f is one of those named fields f_i, and return the value of that field v_i.

In E-RECORD we see that, for a record construction $R(\overline{e}) = R(e_1, \ldots, e_n)$, we must evaluate each e_i to a value v_i (mutating and passing a state along the way) and return a new record $R(\overline{v}) = R(v_1, \ldots, v_n)$ with the last state after each expression has been evaluated σ_n.

In E-LAMBDA we must return a closure. The closure $\mathsf{clo}(\overline{x}, \{\texttt{Ret}\ e;\}, \rho)$ represents everything a function needs to run: the parameters $\overline{x}$, the return statement (a lambda is a function that must immediately return an expression on those parameters with its own arguments), and the environment ρ in which it is created (it can see variables defined before it is created).

In E-APP we first evaluate e, which must give a closure $\mathsf{clo}(\overline{x},\ b,\ \rho_\theta)$ (made by E-LAMBDA, or by P-PROG for a `Func`). We then evaluate each argument e_i to a value v_i from left to right, passing the store along as in E-RECORD, ending with σ_n. Each parameter needs somewhere to live, so we pick fresh locations $\overline{\ell}$ that are not already in σ_n; because every call gets new locations, a recursive call cannot overwrite its caller's parameters. Finally we run the body b in the closure's environment ρ_θ extended with $\overline{x} \mapsto \overline{\ell}$, and in the store σ_n extended with $\overline{\ell} \mapsto \overline{v}$. The body runs in ρ_θ, not the caller's ρ, so it sees the variables from where the function was written rather than where it is called (static scoping). The body must finish with $\mathsf{ret}\ v$: this is the one place an outcome is turned back into a value, and v with the final store σ' is the result of the call.

We continue with the statement rules:

$$\frac{\langle e, \rho, \sigma\rangle \Downarrow \langle v, \sigma'\rangle \qquad \ell \notin \mathrm{dom}(\sigma')}{\langle \texttt{Var}\ x : \tau = e, \rho, \sigma\rangle \Downarrow \langle \mathsf{normal}, \rho[x \mapsto \ell], \sigma'[\ell \mapsto v]\rangle}\ \text{S-VAR}$$

$$\frac{\langle e, \rho, \sigma\rangle \Downarrow \langle v, \sigma'\rangle \qquad \rho(x) = \ell}{\langle x := e, \rho, \sigma\rangle \Downarrow \langle \mathsf{normal}, \rho, \sigma'[\ell \mapsto v]\rangle}\ \text{S-ASSIGN}$$

$$\frac{\langle e, \rho, \sigma\rangle \Downarrow \langle \texttt{False}, \sigma'\rangle}{\langle \texttt{While}\ (e)\ b, \rho, \sigma\rangle \Downarrow \langle \mathsf{normal}, \rho, \sigma'\rangle}\ \text{S-WHILE-F}$$

$$\frac{\langle e, \rho, \sigma\rangle \Downarrow \langle \texttt{True}, \sigma'\rangle \qquad \langle b, \rho, \sigma'\rangle \Downarrow \langle \mathsf{normal}, \sigma''\rangle \qquad \langle \texttt{While}\ (e)\ b, \rho, \sigma''\rangle \Downarrow \langle r, \rho, \sigma'''\rangle}{\langle \texttt{While}\ (e)\ b, \rho, \sigma\rangle \Downarrow \langle r, \rho, \sigma'''\rangle}\ \text{S-WHILE-T}$$

$$\frac{\langle e, \rho, \sigma\rangle \Downarrow \langle \texttt{True}, \sigma'\rangle \qquad \langle b, \rho, \sigma'\rangle \Downarrow \langle \mathsf{ret}\ v, \sigma''\rangle}{\langle \texttt{While}\ (e)\ b, \rho, \sigma\rangle \Downarrow \langle \mathsf{ret}\ v, \rho, \sigma''\rangle}\ \text{S-WHILE-RET}$$

$$\frac{\langle e, \rho, \sigma\rangle \Downarrow \langle \texttt{True}, \sigma'\rangle \qquad \langle b_1, \rho, \sigma'\rangle \Downarrow \langle r_1, \sigma''\rangle}{\langle \texttt{If}\ (e)\ b_1\ \texttt{Else}\ b_2, \rho, \sigma\rangle \Downarrow \langle r_1, \rho, \sigma''\rangle}\ \text{S-IF-T}$$

$$\frac{\langle e, \rho, \sigma\rangle \Downarrow \langle \texttt{False}, \sigma'\rangle \qquad \langle b_2, \rho, \sigma'\rangle \Downarrow \langle r_2, \sigma''\rangle}{\langle \texttt{If}\ (e)\ b_1\ \texttt{Else}\ b_2, \rho, \sigma\rangle \Downarrow \langle r_2, \rho, \sigma''\rangle}\ \text{S-IF-F}$$

$$\frac{\langle e, \rho, \sigma\rangle \Downarrow \langle v, \sigma'\rangle}{\langle \texttt{Ret}\ e, \rho, \sigma\rangle \Downarrow \langle \mathsf{ret}\ v, \rho, \sigma'\rangle}\ \text{S-RET}$$

A reminder that a statement can finish normally with normal or return a value with $\mathsf{ret}\ v$.

In S-VAR we evaluate an expression e to a value v, define a fresh location $\ell \notin \mathsf{dom}(\sigma')$ and store the value there. This is notable as the only statement (or rule) that mutates the environment ρ.

In S-ASSIGN we see a similar structure, only with a mutated store, getting the location to store at through ρ. ρ is unchanged in this rule: the variable must already be declared.

There are three cases for while loops: where the condition is false, or true, or the loop returns early with a $\mathsf{ret}\ v$ statement. S-WHILE-T is recursive, after running the body once, it evaluates to the same statement with a mutated state. S-WHILE-RET returns from the loop early, and passes that value down. Note that the body is a block, its inner environment is thrown away, so that any variable declared in the block is discarded after running.

In the S-IF statements, the condition is evaluated first, then the correct block is selected to be evaluated. The outcome of the block is passed on, so that a `Ret` escapes the `If`.

When a statement returns, we evaluate the expression to a value v and end with a result $\mathsf{ret}\ v$.

The block rules:

$$\frac{}{\langle \{\ \}, \rho, \sigma\rangle \Downarrow \langle \mathsf{normal}, \sigma\rangle}\ \text{B-EMPTY}$$

$$\frac{\langle s_1, \rho, \sigma\rangle \Downarrow \langle \mathsf{normal}, \rho', \sigma'\rangle \qquad \langle \{\ \overline{s}\ \}, \rho', \sigma'\rangle \Downarrow \langle r, \sigma''\rangle}{\langle \{s_1; \overline{s};\}, \rho, \sigma\rangle \Downarrow \langle r, \sigma''\rangle}\ \text{B-SEQ}$$

$$\frac{\langle s_1, \rho, \sigma\rangle \Downarrow \langle \mathsf{ret}\ v, \rho', \sigma'\rangle}{\langle \{s_1; \overline{s};\}, \rho, \sigma\rangle \Downarrow \langle \mathsf{ret}\ v, \sigma'\rangle}\ \text{B-RET}$$

Clearly an empty block simply returns a normal outcome.

B-SEQ takes a list of statements $\overline{s}$ and extracts the first, s_1. If s_1 returns a normal outcome, we take the rest of the block (this can be the empty block), and evaluate that.

B-RET is how a block returns early, we see that the remaining statements $\overline{s}$ are discarded and not run, returning just ret v.

$$\frac{\begin{array}{c}\text{funcs}(p) = \texttt{Func}\ F_j(\overline{x_j : \tau_j}) : \tau'_j = b_j \quad (1 \le j \le k) \qquad \text{stmts}(p) = \overline{s} \\ \Delta = \{R \mapsto (\overline{f : \tau}) \mid \texttt{Record}\ R\ (\overline{f : \tau}) \in p\} \\ \ell_1, \ldots, \ell_k \text{ distinct} \qquad \rho_g = [F_1 \mapsto \ell_1, \ldots, F_k \mapsto \ell_k] \\ \sigma_0 = [\ell_1 \mapsto \mathsf{clo}(\overline{x_1}, b_1, \rho_g), \ldots, \ell_k \mapsto \mathsf{clo}(\overline{x_k}, b_k, \rho_g)] \\ \langle \{\overline{s}\}, \rho_g, \sigma_0\rangle \Downarrow \langle \mathsf{normal}, \sigma'\rangle\end{array}}{\langle p\rangle \Downarrow \langle \sigma'\rangle}\ \text{P-PROG}$$

The master program rule. Going through all the premises:

1. $\text{funcs}(p) = \texttt{Func}\ F_j(\overline{x_j : \tau_j}) : \tau'_j = b_j \quad (1 \le j \le k)$

   We take all the k functions of the program p and index them. We call the name of the function F_j.

2. $\text{stmts}(p) = \overline{s}$

   We take all the n statements of the program p, in order, as a list $\overline{s} = s_1, \ldots, s_n$.

3. $\Delta = \{R \mapsto (\overline{f : \tau}) \mid \texttt{Record}\ R\ (\overline{f : \tau}) \in p\}$

   We build the record table from the record declarations. Records have no run-time behaviour (they create no locations and no values), so this is the only place they appear. Record values store their fields by position only, so E-FIELD uses Δ to find the position of a field from its name.

4. $\ell_1, \ldots, \ell_k$ distinct

   We choose one new location for each function. The store is empty at this point, so they only need to differ from each other.

5. $\rho_g = [F_1 \mapsto \ell_1, \ldots, F_k \mapsto \ell_k]$

   The global environment ρ_g binds each function name to its location. Functions are values, so a function name is looked up like any other variable with E-VAR.

6. $\sigma_0 = [\ell_1 \mapsto \mathsf{clo}(\overline{x_1}, b_1, \rho_g), \ldots, \ell_k \mapsto \mathsf{clo}(\overline{x_k}, b_k, \rho_g)]$

   The initial store holds a closure for each function. Every closure captures ρ_g itself, so every function can see every other function, including itself. This gives recursion, mutual recursion and calls to functions declared later in the program (*hoisting*). Since ρ_g contains only functions, a function cannot see top-level variables.

7. $\langle \{\overline{s}\}, \rho_g, \sigma_0\rangle \Downarrow \langle \mathsf{normal}, \sigma'\rangle$

   Finally, the top-level statements are run as a single block in ρ_g and σ_0. The block must finish with normal, so a top-level `Ret` has no rule and the program gets stuck. The result of the program is the final store σ'.

Note on AI usage: AI (Claude) was used throughout this section as a tutor and checker. It proposed the environment/store model, the shape of the judgements and the normal/ret outcomes; I wrote the expression, block and statement rules, which it then corrected (notably the premises of E-FIELD and E-RECORD, the store passing in B-SEQ, and the outcome in S-WHILE-T). E-LAMBDA's closure structure and the premises of E-APP were developed with its help, and S-VAR, S-ASSIGN, S-WHILE-RET, P-PROG and the explanation of E-APP were written by it. It also proposed the record table Δ (its notation entry and its use in E-FIELD and P-PROG) and wrote the explanation of P-PROG. It also fixed typesetting errors.

4 Implementation

4.1 Architecture

4.2 Correspondence with the Formal Semantics

5 Distinctive-Feature Example Program

6 Reflection on AI and Collaboration

A Remaining Operational Semantics Rules